

Media Upload Flow Review

From the Home Feature's Perspective

Prepared by: Light Bekete

September 29, 2026

Purpose. This document describes how media (photos) are uploaded, labelled, and consumed across the AccoFinder app, with emphasis on the **Home** feature which reads media for the property grid and the property/room detail pages. It documents the current flow, identifies the observed behaviour, and describes where the issue lies. It does not propose solutions — it presents the problem in plain language for review.

1 Overview

Media is uploaded from the phone gallery when an agent or landlord creates a property through the Add Property wizard. Each photo must be assigned to either the whole property or a specific room. Once uploaded, the photo is stored on the backend with a label that identifies which scope it belongs to. On the read side, the Home feature and the Property Detail page consume these photos and decide, based on the label, whether each photo appears in the property's main gallery or in the gallery of a specific room.

The label on each photo is what determines everything downstream. If the label is wrong, the photo appears in the wrong place — regardless of what the user intended when uploading.

2 The Components Involved

The upload flow involves five components. The read flow involves three.

2.1 Upload Side

Component	Role
Property Photos Page	The photo picker inside the Add Property wizard. The user selects photos and assigns each one to a room using a dropdown.
Add Properties Page	Builds the final payload that will be submitted, including the list of photos with their assigned room labels.
Add Property Screen	Submits the property, then iterates the payload's photos and hands each one to the media uploader. Translates the wizard's local room number to the real backend room identifier.
Media View Model	Uploads the photo file to storage, then records the photo on the backend with its label.
Backend Media Store	Stores one record per photo. Each record carries the property it belongs to, the URL, whether it is the primary photo, and the room label.

2.2 Read Side

Component	Role
Media View Model (read)	Returns the list of media records for a given property.
Properties Model	Builds the cover image and the ordered image list for the property cards shown on the Home grid.
Property Details Model	Builds the property's main gallery and the per-room galleries from the same list of media records.

3 The Labelling Contract

A media record's scope is determined **solely** by a single field. There is no other field that distinguishes a whole-property photo from a room photo, and there is no separate collection for each scope.

Three states are possible for this field:

- A special sentinel value that means *this photo belongs to the whole property*.
- A real backend room identifier that means *this photo belongs to that specific room*.
- A legacy positional number that some older records still carry. On the read side, this is treated as equivalent to the room at that position in the property's room list.

Every photo carries exactly one of these values. The read side uses it to decide whether the photo shows up in the property gallery or in a room's gallery.

4 The Upload Flow, in Plain Language

4.1 Picking the Photos

The user opens the Add Property wizard and reaches the Photos step. They pick one or more photos from the device gallery or take them with the camera. At this moment, every picked photo is assigned the sentinel value that means *whole property*. None of them is yet assigned to a room.

The very first photo picked in the batch is also flagged as the primary photo. The others are not. This primary flag is separate from the room assignment and has no effect on which gallery the photo will end up in.

4.2 Assigning Photos to Rooms

Below the picker, the wizard shows a dropdown next to each picked photo. The dropdown lists the whole-property option first, followed by each room the user has added. When the user selects a room for a photo, the wizard updates that photo's label from the sentinel value to the room's local number in the wizard. The local number is a small integer such as 1, 2, or 3 — it is the wizard's own numbering, not the backend's real room identifier.

Each photo has its own dropdown, and each dropdown is independent. In principle the user can assign any subset of the photos to any subset of the rooms.

4.3 Building the Payload

When the user proceeds to review and submit, the wizard builds a payload. The payload contains a list of rooms and a list of photos. For each photo, the payload carries the file path, the primary flag, and the current room label — whatever the wizard's internal model has at that moment.

4.4 Submitting and Translating

The payload is sent to the backend, which creates the property and its rooms and returns the real backend identifiers for each room. Once that response arrives, the upload flow starts iterating the payload's photos. For each photo, it takes the wizard's local room number and translates it to the real backend room identifier by looking the room up in the returned room list. If the translation succeeds, the photo is uploaded with the real room identifier. If the translation fails — for any reason — the photo is uploaded with the sentinel value that means whole property.

This translation step is the last point at which a photo's label can change before it is stored.

4.5 Upload and Persistence

Each photo is uploaded to storage, and then a record is created on the backend that carries the property identifier, the photo URL, the primary flag, and the final room label. From this moment on, the label is fixed and will be read exactly as stored by every screen that consumes it.

5 The Read Flow, in Plain Language

5.1 On the Home Grid

The Home feature displays a grid of property cards. Each card needs a cover image. To build it, the model fetches the media records for that property and picks the record flagged as primary. If no record is flagged as primary, it picks the first record in the list. It does not look at the room label when choosing the cover image. Whichever record is chosen becomes the card's cover.

5.2 On the Property Detail Page

The Property Detail page has two separate galleries that are both built from the same list of media records.

The **whole-property gallery** contains every media record whose label is the sentinel value. These are shown as the main photos of the property.

The **per-room galleries** contain, for each room, every media record whose label matches that room. The read side is tolerant here: it matches the record either by the real backend room identifier or by the legacy positional number. This means it can correctly place photos that were uploaded with either format.

The read side is therefore already as flexible as it can be. It reads the label faithfully and routes the photo to the correct gallery. If a photo shows up in the wrong place, it is because the label stored on the backend is wrong — not because the read side mishandled it.

6 The Observed Problem

During testing of the Home feature, the following behaviour was observed.

When a user assigns multiple photos to the same room in the Add Property wizard, the **first photo is correctly assigned to that room**, but the **remaining photos end up being stored as whole-property photos**. The wizard appears to accept the assignments. The dropdowns appear to show the correct room for each photo. But when the photos are later

read back — either in the property grid or in the property detail page — only the first photo appears in that room’s gallery. The rest appear in the whole-property gallery instead.

This is reproducible: upload three photos, assign all three to a single room, submit, and observe where they end up.

7 Confirming the Problem from the Data

The backend media store was inspected after several uploads. The pattern confirmed the observed behaviour:

- Every property has exactly one media record labelled as whole property and flagged as primary. This is the cover image.
- Properties that should have had multiple photos per room instead show only one photo per room — at most. The remaining photos are not present in any room gallery.
- No property in the sample has two or more photos assigned to the same room, even though the wizard allows it and the user intended it.

In other words, whenever the user assigned more than one photo to the same room, only the first one survived with the correct label. The others were stored with the whole-property sentinel instead.

8 Where the Problem Lies

Because the read side reads the stored label faithfully, and because the translation step that assigns the final label is deterministic — the same input always produces the same output — the problem cannot be in the read side or in the translation step.

The only remaining explanation is that the **wizard’s internal model does not hold the correct room label for the second and subsequent photos at the time the payload is built**. By the time the payload is submitted, those photos are already labelled as whole property. The translation step then sees the sentinel value and correctly stores the sentinel value.

In other words: the wizard’s dropdowns *display* the correct assignment, but that assignment is not reaching the wizard’s model. The model still carries the default sentinel value for photos other than the first, and the payload is built from that model, and the translation step faithfully uploads whatever the payload says.

Two aspects of the wizard’s design make this failure possible.

First, the dropdown’s selected value is bound to the model rather than being set by the user’s action alone. When the model changes for any reason — for example because another photo’s assignment was just updated — the dropdowns re-evaluate. If the underlying dropdown component resets its selection when re-evaluated, that reset can propagate back into the model, silently overriding the user’s earlier choice.

Second, when a dropdown records the user’s selection into the model, it refers to the photo by its position in the list rather than by a stable identifier that travels with the photo. If the list of delegates is re-instantiated for any reason, positions can shift, and an assignment intended for one photo can be written to another — or to a photo that no longer exists. In practice this leads to the assignment being dropped for everything except the first photo.

The model that holds the photos is also declared inside one of the components that displays them, rather than at the top level of the wizard. This ties the model's lifetime to that component's lifetime. If the component is re-created, the model is re-created empty, and any assignments already made are lost.

Taken together, these three characteristics explain the observed behaviour: the first assignment survives because it happens before any re-evaluation; the subsequent assignments do not, because each new assignment triggers a re-evaluation that resets the previously unaffected drop-downs back to the default.

9 What Is Not the Problem

It is worth being explicit about what has been ruled out, so that the person fixing this does not spend time on the wrong areas.

- **The read side is not the problem.** It faithfully reads whatever label is stored, and it already handles both the real backend identifier and the legacy positional number. No change is needed there.
- **The translation step is not the problem.** It is deterministic and returns the same result for the same input. If a photo arrives at the translation step with a real room number, it will be translated correctly. If it arrives with the sentinel value, that is what gets uploaded.
- **The backend storage is not the problem.** It stores exactly what it is given. The media records confirm that the wrong label was already present in the upload request.
- **The C++ view models are not the problem.** They faithfully carry what the QML passes to them. The bug is on the QML side, before the upload request is made.

10 The Impact

Until this is fixed, the Home feature cannot reliably display multiple photos per room. A room that should show three or four photos will show at most one. The remaining photos will appear in the property's main gallery alongside the cover image — which the user did not intend and which makes the property gallery look cluttered. Photographers or agents uploading multiple angles per room will see only the first one routed correctly.

This affects every new property uploaded through the wizard. Older properties that were uploaded before the current wizard was introduced may have their own labelling quirks, but they are not the primary concern — new uploads should be correct.

11 Summary

Media labelling is controlled by a single field on each media record: either a sentinel value meaning whole property, or a room identifier meaning that specific room. The upload flow starts every photo on the sentinel value, allows the user to reassign each photo to a room via a dropdown in the wizard, and records the final assignment when the photo is uploaded. The read flow reads that field and routes the photo to the correct gallery.

A bug exists in the wizard: when multiple photos are assigned to the same room, only the first photo's assignment persists to the model. The remaining photos retain the sentinel value and are uploaded as whole-property photos. This is confirmed by inspecting the backend media store, which shows at most one photo per room per property and exactly one whole-property photo per property — the cover.

The cause is in how the wizard's dropdown interacts with the wizard's internal model: the dropdown's selection is bound to the model in a way that allows re-evaluation to override the user's choice, and photo assignments are recorded against positions rather than stable identifiers. Both of these make the model lose assignments beyond the first.

The read side, the translation step, and the backend are not the cause and do not require changes.

Document prepared by **Light Bekete** for the AccoFinder engineering team.
For questions or hand-off, contact the Home feature owner.